

Observability & Operations - Mentorship Track

Goal: build strong intuition and interview-ready depth for measuring system behavior, debugging production issues, and operating services safely at scale.

How We Will Use This Sheet

- We will keep this sheet focused on 1.9 Observability & Operations.
- We will follow the same learning style used in the asynchronous-system and reliability notes.
- We will add topics one by one in a repeatable architect-level structure.
- We will include code samples, mini programs, and interview-style answers.

Roadmap for This Sheet

1. Metrics, logs, distributed tracing
2. Liveness vs readiness probes
3. SLIs, SLOs, SLAs
4. Canary deployments
5. Blue-green deployments

Topic 1: Metrics, Logs, and Distributed Tracing

Track: 1.9 Observability & Operations Scope: telemetry signals, latency distributions, correlation IDs, trace spans, and production debugging

1. Intuition

Think of observability like diagnosing a patient in a busy hospital.

- Metrics are the vital signs.
- Logs are the detailed notes written during treatment.
- Traces show the patient's full journey through departments.

If the patient is unstable, the vital signs tell you that something is wrong. The notes tell you what happened. The journey tells you where time was spent and where the issue started.

In distributed systems, the same idea applies. You need to know:

- is the system healthy?
- what exactly happened?
- where did the request slow down or fail?

Short memory trick:

- metrics tell you something is wrong
 - logs tell you what happened
 - traces tell you where it happened
-

2. Definition

- Definition: Observability is the ability to understand the internal behavior of a system by analyzing the telemetry it emits.
- Category: Operational visibility and production-debugging discipline
- Core idea: Metrics summarize system behavior over time, logs capture detailed events, and distributed traces connect one request across multiple components.

Classic interview framing:

- metrics, logs, and traces are the three core signals
 - modern observability may also include profiling and events, but the three-signal model is the right baseline
-

3. Why It Exists

Production systems fail in partial, messy ways.

Examples:

- one dependency becomes slow but does not fully fail
- only one endpoint has elevated error rate
- one tenant experiences timeouts while the rest of the fleet looks healthy
- one request crosses six services and the bottleneck is not obvious

Without observability, incident response becomes guesswork.

Teams need observability because they must:

- detect failures quickly
- localize the failing component
- understand why the failure happened
- validate whether a release made things better or worse
- measure user-impacting reliability over time

Simple uptime checks are not enough.

A service can be "up" while users still experience:

- high latency
- partial failures
- queue buildup
- stale data
- degraded dependencies

Observability exists because real systems need evidence, not intuition alone.

4. Reality

Metrics, logs, and tracing are common in:

- microservices platforms
- payment systems
- e-commerce checkouts
- search and recommendation stacks
- data pipelines
- Kubernetes-based applications
- multi-region SaaS platforms

Common tooling examples

- Metrics: Prometheus, CloudWatch, Datadog, Grafana
- Logs: ELK, OpenSearch, Loki, Splunk, CloudWatch Logs
- Tracing: OpenTelemetry, Jaeger, Zipkin, Tempo, X-Ray

Real-world architecture truth

Collecting telemetry is easy. Making it useful is hard.

Teams often fail because they have:

- too many dashboards and no clear signal
- logs without structure
- metrics with bad labels
- traces that break at service boundaries
- alerts that page constantly but explain nothing

Another important truth:

- observability should not depend on human memory

If the system only becomes diagnosable after an engineer manually adds a temporary log line during an incident, the instrumentation is too weak.

5. How It Works

At a high level:

1. A request enters the system.
2. The service creates or propagates a trace ID and span context.
3. The service emits metrics for request rate, errors, and latency.
4. The service writes structured logs with request context.
5. The service creates trace spans around internal work and downstream calls.
6. Telemetry is exported to storage and visualization systems.
7. Operators use dashboards, alerts, logs, and traces to investigate behavior.

Metrics flow

- Counters track totals such as requests, errors, retries, or queue events.
- Gauges track current values such as queue depth, CPU, memory, or active connections.
- Histograms or summaries track latency distributions so teams can reason about p95 and p99.

Metrics answer questions like:

- Is traffic up or down?
- Is error rate increasing?
- Is latency getting worse?

Logs flow

- The service writes event records when meaningful things happen.
- Good logs are structured, searchable, and contextual.
- Important fields often include service name, trace ID, request ID, endpoint, dependency, status, and error code.

Logs answer questions like:

- What exactly failed?
- Which order, user flow, or dependency was involved?
- What exception or business event occurred?

Distributed tracing flow

- One request gets a trace ID.
- Each service call creates a child span.
- Spans record duration, status, metadata, and parent-child relationships.
- Trace context is propagated across HTTP, gRPC, messaging, or async boundaries.

Tracing answers questions like:

- Which hop was slow?

- Did latency come from the database, cache, or external API?
- How did this request move across services?

Failure path

- If the telemetry backend is slow or unavailable, the application should fail open.
- Telemetry export should be buffered, batched, sampled, or dropped before it blocks user requests.
- Observability failure should not become application failure.

Recovery path

- Exporters retry or reconnect.
 - Buffered telemetry drains when the backend recovers.
 - Dashboards and alerts resume, and traces become queryable again.
-

6. What Problem It Solves

- Primary problem solved: detects, localizes, and explains production failures and performance regressions
- Secondary benefits: capacity planning, release verification, trend analysis, faster incident response, and stronger postmortems
- Systems impact: changes operations from guess-and-check debugging to evidence-driven diagnosis

Observability is what lets a team answer:

- what is broken?
 - where is it broken?
 - how bad is it?
 - is it getting worse?
-

7. When to Rely on It

Use strong observability when:

- the system is customer-facing
- multiple services participate in one user request
- latency and error budgets matter
- the team runs on-call rotations
- production changes must be validated safely
- dependencies can fail partially or intermittently

Especially important for:

- microservices
- asynchronous pipelines

-
- payment flows
 - search systems
 - SaaS control planes
 - high-traffic APIs

Strong interviewer keywords:

- p99 latency
 - root cause
 - request path
 - correlation ID
 - dependency bottleneck
 - noisy alerting
 - incident triage
-

8. When Not to Use It

Do not instrument blindly.

Be careful when:

- adding high-cardinality metric labels such as user ID or order ID
- logging secrets, tokens, passwords, or sensitive PII
- tracing every successful request at full fidelity in very high-volume systems
- turning debug logging on globally in a hot code path

Also avoid this misunderstanding:

- more telemetry is not automatically better telemetry

Poor observability can create:

- high cost
- storage pressure
- query slowdown
- alert fatigue
- harder debugging because signal is buried in noise

Better framing:

- use metrics for aggregate health
 - use logs for rich event detail
 - use traces for request-path analysis
 - choose sampling, retention, and cardinality deliberately
-

9. Pros and Cons

Pattern	Pros	Cons
Metrics, logs, and tracing	Fast detection, strong drill-down debugging, SLO measurement, and better release confidence	Storage cost, instrumentation effort, noisy signals if poorly designed, and cardinality or retention trade-offs

10. Trade-offs and Common Mistakes

Trade-offs

- Detail vs cost: richer telemetry improves diagnosis but increases ingest, storage, and query cost.
- Aggregation vs precision: metrics are cheap and fast to query, but logs and traces hold deeper detail.
- Full-fidelity tracing vs overhead: tracing every request helps debugging, but sampling is often needed at scale.
- Flexibility vs safety: free-form logs are easy to write, but structured logs are much easier to query reliably.
- Rich dimensions vs cardinality risk: labels help analysis until they explode the metric store.

Common Mistakes

Mistake	Why it is wrong	Better approach
Using user IDs as metric labels	Creates unbounded cardinality and can overwhelm the metrics backend	Keep labels bounded; use logs or traces for request-specific identity
Relying only on averages	Averages hide tail latency and user pain	Track p50, p95, and p99 latency
Writing unstructured logs	Hard to filter, correlate, and automate	Use structured logs with stable fields
Forgetting trace propagation across services	Distributed traces break into disconnected fragments	Standardize context propagation using OpenTelemetry or consistent middleware
Making telemetry export part of the critical path	Observability outages can harm application availability	Fail open and export asynchronously
Logging sensitive data	Creates compliance, security, and privacy risk	Redact or avoid secrets and sensitive identifiers

11. Key Numbers

These are practical heuristics, not universal laws.

- Rate, errors, duration: the RED method is a strong baseline for request-driven systems.
- p50, p95, p99 latency: p95 and p99 are often more useful than averages for user-facing systems.
- Error rate: teams often alert on sustained error percentages, not just absolute failures.
- Trace sampling rate: high-volume success traffic may be sampled, while error traces are often kept at a much higher rate.
- Log retention: hot searchable retention may be days to weeks, while colder archival retention may be much longer.
- Alert evaluation window: often a few minutes for urgent regressions and longer for slower-burn saturation issues.
- Cardinality budget: metric dimensions should stay bounded and predictable.

Interview shorthand:

- RED metrics, p99, sampling, retention, cardinality, correlation ID
-

12. Failure Modes

High-cardinality metrics

Problem:

- Labels such as session ID, order ID, or user ID create too many unique time series.

User impact:

- dashboards slow down, monitoring cost spikes, and useful metrics become harder to query during incidents

Mitigation:

- keep labels bounded
- move unique identifiers into logs and traces
- review metric cardinality regularly

Broken trace propagation

Problem:

- One or more services fail to forward trace context.

User impact:

- the slow request is visible only as disconnected fragments, so root cause analysis gets much harder

Mitigation:

- standardize middleware
- use automatic instrumentation where possible
- test trace continuity across sync and async boundaries

Noisy alerting

Problem:

- alerts fire on low-value signals or transient noise.

User impact:

- operators learn to ignore pages, and real incidents are detected late

Mitigation:

- alert on actionable symptoms
- tie alerts to user impact or SLO risk
- tune thresholds and suppression windows

Telemetry backend outage

Problem:

- logging, metrics, or tracing infrastructure becomes degraded or unavailable.

User impact:

- teams lose visibility during an incident, which slows diagnosis

Mitigation:

- fail open in the application
 - batch and buffer exporters
 - monitor the observability stack itself
-

13. Scenario

- Product / system: Checkout platform with API gateway, checkout service, payment service, inventory service, and PostgreSQL
 - Requirement: on-call engineers must quickly diagnose intermittent checkout latency spikes and payment failures
 - Good design: RED metrics on every service and dependency, structured logs with trace IDs and order IDs, and distributed traces across API, payment, inventory, and database calls
 - Why this concept fits: the failure path crosses service boundaries, so no single service log is enough
 - What would go wrong without it: engineers would know checkout is slow, but not whether the cause is payment latency, database saturation, or queue buildup
-

14. Code Sample

Emitting all three signals from one request path

```
public OrderResponse placeOrder(OrderRequest request) {
```

```

Span span = tracer.spanBuilder("checkout.place_order").startSpan();
long startNanos = System.nanoTime();

try (Scope ignored = span.makeCurrent()) {
    String traceId = span.getSpanContext().getTraceId();

    logger.info("event=place_order_start traceId={} orderId={}",
        traceId,
        request.orderId());

    OrderResponse response = paymentClient.charge(request);

    meterRegistry.counter("checkout.requests.total", "outcome", "success").increment();
    return response;
} catch (Exception ex) {
    meterRegistry.counter("checkout.requests.total", "outcome", "error").increment();
    span.recordException(ex);
    span.setStatus(StatusCode.ERROR);

    logger.error("event=place_order_failed traceId={} orderId={} message={}",
        span.getSpanContext().getTraceId(),
        request.orderId(),
        ex.getMessage());

    throw ex;
} finally {
    meterRegistry.timer("checkout.request.latency")
        .record(System.nanoTime() - startNanos, TimeUnit.NANOSECONDS);
    span.end();
}
}

```

Key idea:

- one request should emit aggregate health signals, detailed event records, and end-to-end request context

15. Mini Program / Simulation

This mini program shows one request producing logs, metrics, and a simple trace.

```

import time
from collections import Counter, defaultdict
from uuid import uuid4

metrics = Counter()
latencies = defaultdict(list)

def log(level: str, event: str, **fields) -> None:
    record = {"level": level, "event": event, **fields}
    print(record)

class Span:

```

```

def __init__(self, trace_id: str, name: str, parent: str | None = None) -> None:
    self.trace_id = trace_id
    self.name = name
    self.parent = parent
    self.start = time.perf_counter()
    log("INFO", "span_start", traceId=trace_id, span=name, parent=parent)

def finish(self, error: str | None = None) -> None:
    duration_ms = round((time.perf_counter() - self.start) * 1000, 2)
    metrics[f"{self.name}.count"] += 1
    latencies[self.name].append(duration_ms)

    if error:
        metrics[f"{self.name}.errors"] += 1

    log(
        "ERROR" if error else "INFO",
        "span_finish",
        traceId=self.trace_id,
        span=self.name,
        durationMs=duration_ms,
        error=error,
    )

def inventory_call(trace_id: str) -> None:
    span = Span(trace_id, "inventory.reserve", parent="checkout.place_order")
    time.sleep(0.02)
    span.finish()

def payment_call(trace_id: str) -> None:
    span = Span(trace_id, "payment.charge", parent="checkout.place_order")
    time.sleep(0.08)
    span.finish(error="gateway_timeout")

def place_order(order_id: str) -> None:
    trace_id = str(uuid4())
    root = Span(trace_id, "checkout.place_order")

    log("INFO", "request_start", traceId=trace_id, orderId=order_id)
    metrics["checkout.requests.total"] += 1

    inventory_call(trace_id)
    payment_call(trace_id)

    metrics["checkout.requests.error"] += 1
    log("ERROR", "request_failed", traceId=trace_id, orderId=order_id,
        reason="gateway_timeout")
    root.finish(error="gateway_timeout")

def main() -> None:
    place_order("order-123")

    print("\nMetrics summary")
    for name, value in metrics.items():
        print(name, value)

```

```
print("\nLatency summary")
for name, values in latencies.items():
    print(name, values)

if __name__ == "__main__":
    main()
```

What this demonstrates:

- the same trace ID ties together events across spans
- metrics summarize what happened
- logs hold contextual detail for the failing request
- traces expose where latency and failure occurred

16. Practical Question

You are designing a checkout platform with an API gateway, checkout service, payment service, inventory service, and database. How would you instrument metrics, logs, and distributed tracing so the on-call team can diagnose a p99 latency spike quickly?

17. Strong Answer

I would absolutely instrument all three signals, because a latency spike across multiple services is exactly the kind of problem that one signal alone explains poorly.

I would start with metrics using the RED model at each service boundary: request rate, error rate, and latency distributions by endpoint and by dependency. That gives the team fast detection and lets them see whether the spike is isolated to checkout, payment, database access, or a specific external dependency.

Then I would add structured logs with stable fields such as service name, trace ID, request ID, endpoint, dependency, status code, and business-safe identifiers like order ID. I would avoid putting unbounded identifiers into metrics, and I would avoid logging secrets or sensitive payloads.

For distributed tracing, I would propagate trace context end to end using standard instrumentation such as OpenTelemetry. Each service hop and dependency call would produce spans so the team can see whether time is being spent in application logic, a database query, a remote payment call, or queue wait.

At scale, I would sample successful traces but keep a much higher rate for errors and slow requests. If the system were still small and mostly monolithic, I might start with strong metrics and structured logs first, then add full tracing once request paths become more distributed.

18. Revision Notes

- One-line summary: Observability turns production debugging from guesswork into evidence by combining metrics for trends, logs for events, and traces for request flow.

Three keywords: RED, correlation ID, spans

- One interview trap: putting unbounded values like user ID into metric labels
- One memory trick: metrics tell you something is wrong, logs tell you what happened, traces tell you where it happened